DSA 101 to HackerRank contests

How to turn the curriculum spreadsheet into two HackerRank contests — one for classwork, one for homework.

What you are working with

Everything lives in `C:\Users\sunst\Desktop\hackerrank contest`. A handful of Python scripts read one sheet of your curriculum workbook — `DSA_101` here — help you map each exercise to a HackerRank challenge, and then build the two contests in your account.

Where this stands

Both contests exist and are verified in sync: **DSA 101 - Classwork** (id 394561) holds 105 challenges and **DSA 101 - Homework** (id 394562) holds 71. Together that covers 176 of the 299 problems in the sheet.

The other 123 are not a bug. 70 have no similar challenge in either library and need one authored by hand; 24 are paper or reading exercises no judge can grade; 29 share a challenge with an earlier problem. `DSA101_Unmapped_Questions.pdf` lists all of them by reason.

Two courses share these scripts

This guide walks through DSA 101, but the same pipeline builds DSA 301 from the `DSA_301` sheet of the same workbook. Every command below takes `--course 301` on either side of the subcommand; without it you get DSA 101. `courses.py` holds the definitions — which sheet, which files, which contests and which slice of lectures each course means.

	DSA 101	DSA 301
Sheet	DSA_101	DSA_301
Scope	Day 1-60	LEC 1-50
Mapping files	classwork.md homework.md	301-classwork.md 301-homework.md
Contest ids	394561 / 394562	394608 / 394610
Challenges live	105 / 71	138 / 145
Gap report	DSA101_Unmapped_Questions.pdf	DSA301_Unmapped_Questions.pdf

DSA 301 runs on a different HackerRank account

A contest can only hold challenges the signed-in account can see, and the DSA 301 challenges were authored under **experience1**. So switching course also means switching login: `reset-login`, then `login`, then `whoami` to confirm before anything writes, then `sync-mine` to re-cache the bank.

A bank cached from the wrong account maps every problem to a challenge the target account cannot add, and the failure looks like a broken endpoint rather than a wrong cache. `whoami` costs a second and rules that out.

DSA 301 was far easier to map than DSA 101: its account authored the challenge bank from these very spreadsheet cells, so 295 of the 300 problems matched with a worst score of 0.87. Five need authoring by hand. All of that is in `DSA301_Unmapped_Questions.pdf`.

The files

File	What it is
`courses.py`	Which sheet, files, contests and lecture range each --course means.
`extract_curriculum.py`	Reads the .xlsx and writes the two markdown lists.
`classwork.md` `homework.md`	**The mapping you edit.** One row per problem: ID, problem text, Slug, Type.
`hr_index.py`	Caches HackerRank's public challenge library locally.
`resolve_slugs.py`	Fuzzy-matches problems; writes the slug_candidates.md review sheet.
`md_table.py`	Shared table parser, so every script reads the markdown identically.
`create_contests.py`	Logs you in, creates the contests, adds the challenges.
`make_gaps_pdf.py` `pdf_style.py`	Builds the coverage-gap report PDF for whichever course you pass.

Generated files you can delete at any time: `challenge_index.json`, `my_challenges.json`, `slug_candidates.md`, `discovered_endpoints.json`, `apply_log.json`, `.hr_profile/`, and the `301-` prefixed copies of each. Every generated file except the challenge index and the bank cache is course-scoped, so the two courses never overwrite each other's review sheets or logs.

One-time setup

Open PowerShell and move into the project folder:

```
cd "C:\Users\sunst\Desktop\hackerrank contest"
```

Install the dependencies:

```
pip install openpyxl playwright
```

That is all. The scripts drive the Google Chrome already installed on this machine, so there is no separate browser download. Chrome is launched against its own profile directory, `.hr_profile/`, not your everyday profile — so your normal browsing session, and any other HackerRank account you are signed into, are left untouched.

1 Generate the two problem lists

Already done, so run this only if you change the spreadsheet. `--merge` rewrites both files from the sheet while keeping every slug you have already filled in.

```
python extract_curriculum.py --merge
```

Expected output:

```
read 60 lecture rows from DSA_101_Curriculum.xlsx
wrote classwork.md: 179 problems (164 code, 15 theory)
wrote homework.md: 120 problems (111 code, 9 theory)
```

Plain `extract_curriculum.py` refuses to overwrite existing files. `--force` regenerates from scratch and discards your slugs.

2 Log in to HackerRank

A Chrome window opens. Sign in yourself, using the account that should own these contests. The script never sees or handles your password — only the resulting session cookie is stored locally. Press Enter in the terminal once your dashboard is visible.

```
python create_contests.py login
```

Confirm which account the stored session belongs to at any time:

```
python create_contests.py whoami
```

To sign in as a different account later, run `python create_contests.py reset-login` and log in again. To use a different browser: `--browser msedge` or `--browser chromium` (that one needs `python -m playwright install chromium` first).

3 Match against your own authored challenges

This is the step most likely to save you days of work. If you have already authored these problems as challenges on this account, they live in a private library that the public search cannot see. Pull it, then match against it:

```
python create_contests.py sync-mine
```

```
# preview the matches without changing anything
python create_contests.py match-mine

# write the confident ones into classwork.md and homework.md
python create_contests.py match-mine --apply
```

If `sync-mine` reports a healthy challenge count, most of the mapping resolves itself here. If it reports none, go to Step 4.

4 Fill in the remaining slugs

A slug is the last path segment of a challenge URL:

```
https://www.hackerrank.com/challenges/simple-array-sum/problem
                                      ^^^^^^^^^^^^^^^^
```

Build the candidate review sheet, which lists the top four public-library matches per unmapped problem with scores and clickable URLs:

```
python resolve_slugs.py
```

Open `slug_candidates.md`, check the ones that look right, and paste their slugs into the Slug column of `classwork.md` and `homework.md`. To let the high-confidence ones write themselves:

```
python resolve_slugs.py --apply

# stricter threshold if you want fewer, safer matches
python resolve_slugs.py --apply --min-score 0.9
```

> **Treat every auto-filled slug as a guess until you have opened it**
>
> The matcher compares wording, not meaning. It confidently proposes `binary-search-tree-insertion` for `"LC704 Binary Search"` and `game-of-stones-1` for `"Sieve of Eratosthenes"`. Both are wrong, and both score highly.

Anything with no candidate needs a challenge authored by hand on HackerRank — statement, test cases, reference solution. Nothing here can generate that for you. The workflow is built to be incremental: add slugs as you create challenges, re-run Step 6, and only the new ones get added.

5 Check what will happen

This writes nothing to HackerRank. Run it every time before Step 6.

```
python create_contests.py plan

# add -v to list every challenge that would be added
python create_contests.py plan -v
```

It reports, per contest, how many problems are listed, how many are codeable, how many carry a slug, and how many will be skipped.

6 Create the contests

```
python create_contests.py apply
```

It prints a summary and asks you to type `yes` before touching anything. Re-running it is safe: existing contests are reused, and challenges already present are not added twice — so you can keep filling in slugs and run it again as often as you like. A record of what happened is written to `apply_log.json`.

7 Verify, and trust the right number

Counts can agree while contents differ, so compare the sets:

```
python create_contests.py diff
```

It reports two directions: **missing** (mapped but not in the contest, so run `apply`) and **extra** (in the contest but not mapped). It printed IN SYNC once both contests matched exactly.

> **Trust VERIFIED and diff, not the added count**
>
> HackerRank answers a *rejected* write with **HTTP 200** and a body of `{"status": false, "errors": ...}`. Before the scripts checked that field, 53 challenges were reported added while nothing changed. The added counter reflects what the server replied; the `VERIFIED` line re-reads the contest, and `diff` compares the full sets. At one point both contests showed the expected total while classwork was missing one problem and holding one stray - the two errors cancelled out, and only the set comparison caught it.

Changing the mapping later

Edit the Slug column in the markdown directly, or let the matcher fill it. Note that `--best-effort` only fills *empty* cells, so raising `--floor` on its own leaves earlier, weaker slugs in place. `--rescore` re-judges what is already written and clears anything that no longer qualifies.

```
python resolve_slugs.py --best-effort --rescore --floor 0.6 --apply
```

Clearing a slug locally does not remove it from a contest. The rescore writes `removals.json` listing what still needs deleting:

```
python create_contests.py remove          # dry run, shows targets
python create_contests.py remove --go     # actually delete

# or remove anything live that the markdown does not map at all
python create_contests.py remove --extra --go
```

If something goes wrong

A request fails, or succeeds without doing anything

HackerRank publishes no documented contest API. These scripts send the same requests its own web app sends, recorded with `discover` and listed in `ENDPOINTS` at the top of `create_contests.py`. They can change without notice.

Do not guess at a replacement - several wrong guesses preceded every endpoint below. Record the real request instead:

```
python create_contests.py discover
```

Do the operation by hand in the browser it opens. Every write is printed and saved to `discovered_endpoints.json`, with analytics filtered out. Correct `ENDPOINTS` and re-run.

The confirmed requests

Operation	Request, and the part that is easy to get wrong
List contests	GET /rest/administration/contests?offset=&limit= — **paginate.** This account has 851 and the list is ascending by id, so a new contest sorts last.
Contest challenges	GET /rest/administration/contests/{id}/challenges — **numeric id only.** The slug 404s even when it is correct.
Add a challenge	POST /rest/administration/contests/{id}/challenge?slug=&track_id=0&chapter_id=0&weight=5 — **singular /challenge, parameters in the query string, empty body.**
Remove a challenge	DELETE /rest/administration/contests/{id}/association?ca_id=&chapter_id=0 — **keyed by the association id**, read from the contest listing, not the challenge id.
Contest admin page	/administration/contests/edit/{id}/challenges — **the HTML namespace does not mirror the REST one.**

The plural form `POST .../contests/{id}/challenges` is a trap: it *authors a new challenge* and rejects an add payload with `Validation failed: Name can't be blank` - while answering HTTP 200.

Other issues

Symptom	Fix
Playwright is not installed	pip install playwright
no usable browser found	Install Google Chrome, or run python -m playwright install chromium
"not logged in" right after logging in	Do not pass --headless. HackerRank serves headless Chrome a bot-check page, so a valid session looks logged out.
Wrong HackerRank account	python create_contests.py reset-login, then log in again
The window is too small to work in	Already fixed: visible sessions use the real window size and start maximised.
A duplicate contest appeared	apply now looks a contest up before creating it. HackerRank does not reject a duplicate name with a 409; it silently creates a second contest.
A problem is wrongly marked theory	Edit its Type cell to code, or adjust THEORY_PATTERNS in extract_curriculum.py
Your slugs vanished after regenerating	Use --merge, not --force. --force intentionally discards slugs.

Command reference

Command	What it does
`python extract_curriculum.py --merge`	Rebuild both markdown lists from the spreadsheet, keeping slugs.
`python resolve_slugs.py`	Build slug_candidates.md. Writes nothing else.
`python resolve_slugs.py --best-effort --apply --floor 0.5`	Fill every problem whose closest match scores at least 0.50.
`python resolve_slugs.py --best-effort --rescore --floor 0.6 --apply`	Also re-judge slugs already written and clear those below the floor.
`python create_contests.py login`	Open a browser and wait for you to sign in.

Command	What it does
`python create_contests.py status`	Every contest for this course, its challenge count and links.
`python create_contests.py whoami`	Which account the saved session belongs to. Run before any write.
`python create_contests.py import-mine <file>`	Cache an authored-challenge backup from disk, with no login.
`python create_contests.py diff`	Compare the mapping to the live contests, both directions.
`python create_contests.py plan -v`	Show what apply would do. Writes nothing.
`python create_contests.py apply`	Create or reuse the contests and add missing challenges. Asks first.
`python create_contests.py remove`	Dry run of what removals.json would delete. Add --go to do it.
`python create_contests.py remove --extra --go`	Delete anything a contest holds that the markdown does not map.
`python create_contests.py sync-mine`	Cache the challenges this account has authored.
`python create_contests.py discover`	Record the real API requests while you work by hand.
`python make_gaps_pdf.py --course 301`	Rebuild the coverage-gap report PDF.
`python create_contests.py reset-login`	Forget the stored session.

Two things worth knowing

Contest size. 105 challenges in one contest is a lot for students to navigate, and DSA 301's classwork contest holds 138. For one contest per week or per day, or for a third course, `courses.py` is the only thing to change - it drives contest creation, which file each contest reads, and where every generated file lands.

The mapping is approximate. Slugs were assigned by comparing wording against 3,400 challenges with a 0.50 floor. Spot-checks look right, but that is evidence rather than proof - `mapping_report.md` lists every assignment with its score, weakest first, if you want to audit them.